AI 模式 全部 圖片 影片 新聞 更多

提出任何問題

+

在 Lean 4 中使用 **Jixia**（Lean 4 的靜態分析工具 1.2.3）提取定理（如 `lerec_succ_left`）時，若出現「類型範圍不正確」（extract type range not correct）的錯誤，通常與 **依賴參數（Dependent Arguments）** 的處理或 **Jixia 版本相容性**有關。

以下是針對該定理代碼可能導致提取錯誤的原因分析與解決方案：

1. 依賴動機（Motive）導致的類型不正確

在你的定理中，`motive` 是一個依賴類型：`motive : (m : ℕ) → n ≤ m → sort*`。

- **問題原因**：當靜態分析工具嘗試提取或重寫涉及依賴類型的項時，如果重寫了 `m` 但沒有同步更新證明 `n ≤ m`，就會導致 `motive` 的參數類型不一致 1.5.1。這在 Lean 中常表現為 `motive is not type correct` 錯誤 1.5.10。
- **解決方案**：
  - **手動提供 `@lerec` 參數**：你目前的代碼已經使用了 `@lerec` 並填寫了部分下劃線 `_`。嘗試完全顯式地寫出所有參數，這能幫助 Jixia 準確定位每個項的類型範圍。
  - **使用 `simp_rw` 或 `subst`**：如果錯誤發生在 `rw` 步驟，嘗試改用 `simp_rw`，它在處理依賴目標的重寫時通常比標準 `rw` 更健壯 1.1.1。

2. Jixia 與 Lean 版本不匹配

Jixia 對 Lean 版本極其敏感 1.2.3。

- **問題原因**：Jixia 必須使用與被分析檔案**完全相同版本**的 Lean 構建。版本不一致會導致輸出格式出錯或無法正確解析類型範圍 1.4.1。
- **解決方案**：檢查你的 `lean-toolchain` 檔案，確保 Jixia 的構建版本與你的專案版本（目前推薦相容版本為 **v4.16.0**）一致 1.2.3。

AI 模式 全部 圖片 影片 新聞 更多

9 個網站

Homotopy Type Theory in Lean | Springer Nature Link

The elaborator fills in implicit arguments, which can be inferred from the context, such as the type of the term to be constructed...

Springer Nature Link

Topic: mathematics in lean error - Zulip Chat Archive

2026年2月28日 — Jeremy Avigad (Jul 30 2020 at 13:13): This is a quirk of Lean's parser: example is parsed like def rather than theorem , which mea...

Lean community

(CVE-2020-7961)Liferay Portal RCE 反序列化命令执行漏洞 - 知乎专栏

2020年12月19日 — (CVE-2020-7961)Liferay Portal RCE 反序列化命令执行漏洞 - 漏洞描述 Liferay Portal CE是一款用来快速构建网站的…

知乎专栏

顯示全部

here is comparison: old is expected and new is current:
module_name ['Mathlib', 'Data', 'Sign', 'Defs'] is different on decls

這通常表示 **Jixia** 在分析 `Mathlib.Data.Sign.Defs` 模組時，提取出的聲明（Declarations）與預期（Expected）的基準數據不一致。

這種情況通常由以下三個原因引起：

+

量（Start/End offset）與實際文件不符。

3. **依賴項載入順序**：
   如果 `Mathlib.Data.Sign.Defs` 依賴的其他模組被修改過，Lean 的增量編譯可能會導致生成的 `.ilean` 或 `.olean` 文件包含額外的輔助定義（如 `_sunfold` 或生成的 `match` 定理），這會干擾 Jixia 的聲明列表。

建議處理步驟：

- **清理並重新編譯**：
  執行 `lake clean` 然後 `lake build`。這能確保沒有舊的編譯產物干擾提取邏輯。
- **強制更新 Jixia**：
  確保你的 Jixia 工具是用當前專案的 `lean-toolchain` 版本編譯的。
- **對比具體差異**：
  Jixia 通常會輸出具體的 `diff`。檢查是**缺漏了定理**，還是定理的**起始/結束行號**偏移了幾行？如果是行號偏移，通常是源碼中存在多餘的換行或註解干擾了分析。

請問你在執行提取指令時，是否使用了特定的 **Commit ID** 或 **Lean 版本**？

{"path": "[19, Nat.leRec_succ_left].type.range.start", "name": "start", "old": "8403", "new": "8301", "parent_old": "8403", …

這個錯誤顯示 **Jixia** 在提取 `Nat.leRec_succ_left` 定理的類型範圍（type range）時，起始位置（start）發生了 **102 個字元的偏移**（從 8403 變為 8301）。

+

- **偏移原因**：8403 與 8301 之間相差的 102 個字元，很可能對應於該定理前面的 **Doc String（註解）**、**大量隱式參數的展開** 或 **Namespace 的縮排** [3]。

2. 解決方案：對齊 Elaborator 資訊

要修復這個 `range` 不正確的問題，請嘗試以下步驟：

- **簡化 `@` 的調用**：
  嘗試將顯式參數改回隱式，或者使用 `named arguments`（例如 `(motive := motive)`），看偏移是否消失。Jixia 對完全展開的 `@` 語法有時會抓取到錯誤的 Syntax 原型。
- **檢查換行與空行**：
  偏移量 102 字符通常接近 2-3 行代碼。請確認 `Mathlib/Data/Sign/Defs.lean` 文件中該定理上方是否有新加入的註解或 `import` 語句。Jixia 依賴於 `FileMap`，任何微小的格式變動都會導致 `old` 與 `new` 比對失敗 [4]。
- **強制重新同步 InfoTree**：
  在該定理上方加入 `set_option trace.Elab.definition true` 重新編譯，觀察 Lean 輸出中定義的起始點是否真的落在 8301。如果 8301 才是正確的，則需要更新你的 **Expected Data（預期基準）** [2, 5]。

3. 工具層面的修復

如果這是在執行自動化 CI 或比對腳本：

1. **更新基準值**：如果你的代碼邏輯沒變，僅是位置移動，請將 `expected` 更新為 `8301`。
2. **Lake Clean**：執行 `lake clean` 刪除所有編譯緩存（`.olean`, `.ilean`），確保 Jixia 讀取的是最新的源碼映射信息 [1, 4]。

你目前的 `Mathlib` 是透過 `lake update` 更新過的嗎？ 這可能是因為上游文件的格式微調導致的偏移。

+

圖透過硬編碼的子節點索引（如 `realStx[2]`）手動拆解 `Syntax` 樹。

在 Lean 4 中，如果你在 `theorem` 前面加了 `@` 或者有複雜的 `modifiers`（如 `lemma` 的屬性），`Syntax` 的樹狀結構會改變，導致 `realStx[2]` 抓到的不是真正的 `sigStx`（簽名區塊），進而導致後續 `typeStx` 抓到錯誤的偏移量。

核心問題點：

1. **索引偏移**：在 `if normKind == "definition" || ...` 分支中，代碼假定 `realStx[2]` 是簽名。但對於 `lemma` 或帶有特定 `modifiers` 的定義，這個位置可能指向了參數列表的一部分或是關鍵字。
2. **`typeSpec` 的抓取邏輯**：代碼使用 `sigStx[sigStx.getNumArgs - 1]` 獲取類型。如果簽名解析不全，這會直接抓到錯誤的語法節點。

修復方案：

不要使用 `stx[n]` 這種硬編碼索引，改用 Lean 核心提供的 **`TSyntax` 模式匹配**，這能確保無論語法如何變化（有無 `@`，有無屬性），都能精確定位到 `type` 部分。

以下是針對 `theorem/definition` 部分的修復建議：

lean

```
-- 修改後的 (sigStx, valueStx) 提取邏輯
let (sigStx, valueStx) ← match realStx with

  | `(Lean.Parser.Command.declaration| $[$modifiers?]? theorem
      pure (sig, val)

  | `(Lean.Parser.Command.declaration| $[$modifiers?]? def $id
      pure (sig, val)
```

+

```
  -- 處理手動拆解
  if typeSpec.getNumArgs > 1 && (typeSpec[0].reprint.getD "")
    typeSpec[1]
  else
    typeSpec
```

請謹慎使用程式碼。

為什麼這能修復 102 字元的偏移？

- **精確定位**：`8403` 到 `8301` 的偏差通常是因為 `typeData.range` 抓取到了包含冒號 `:` 甚至包含前面參數的節點。
- **跳過裝飾器**：使用 `match` 可以自動處理 `modifiers` 和 `declId` 佔用的長度，直接將 `type` 定位在真正的類型表達式起始處。

建議操作：

1. 將 `sigStx` 和 `valueStx` 的賦值邏輯改為基於 **`match realStx with`**。
2. 在提取 `typeData` 之前，檢查 `typeStx.getRange?`。如果抓到的是 `:`，則偏移量一定會出錯，應取其後一個節點。

您是否正在使用 **Mathlib** 的 `lemma` 關鍵字？如果是，請確保您的 `match` 語法也包含對 `lemma` 的處理，因為它在 `Syntax` 樹中與 `theorem` 是不同的節點類型。

problem is: before type, it has comment and looks not correct to

+

核心問題分析

1. **註解干擾判斷**：你的代碼 `let firstChildPP := (typeSpec[0].reprint.getD "").trim` 雖然用了 `.trim`，但如果註解是在節點內部或作為 `Trivia` 存在，`typeSpec[0]` 可能抓取到的是包含註解的整個區塊，導致它不等於 `":"`。
2. **範圍抓取過大**：當 `firstChildPP == ":"` 判斷失敗時，代碼會執行 `else typeSpec`，這會把**整個類型宣告（包含冒號、註解、空格）**都當作類型範圍，這就是為什麼你的 `start` 偏移了 102 個字元（正好是註解的長度）。

修復方案

建議使用 `Lean.Syntax.asControls` 或更穩健的子節點導航來跳過 `Trivia` 和註解。請修改提取 `typeStx` 的邏輯如下：

lean

```lean
-- 1. 定義一個輔助函數來獲取真正的起始位置（跳過冒號和其前後的註解）
let typeStx :=
  if typeSpec.isOfKind ``Lean.Parser.Term.typeSpec && typeSpec.
    -- index 0 是 ":"，index 1 才是真正的類型表達式
    typeSpec[1]
  else if typeSpec.getNumArgs > 1 then
    -- 原有的邏輯，但增加對第 0 個節點是否為冒號的穩健檢查
    let firstChild := typeSpec[0]
    if firstChild.isAtom && firstChild.getAtomVal.trim == ":" t
      typeSpec[1]
    else
      typeSpec
  else
    typeSpec
```

+

為什麼這樣能修復？

- **避開 `reprint` 判斷**：`firstChild.getAtomVal` 比 `reprint.trim` 更可靠，因為 `AtomVal` 只包含該標記（Token）本身（即 `:`），而不包含其關聯的註解（Trivia）。
- **精確索引**：`typeSpec[1]` 直接指向類型定義的語法樹節點。在 Lean 的解析器中，`typeSpec` 的結構固定為 `[":", "Term"]`。即使冒號後有長篇大論的註解，`typeSpec[1].getRange?` 的 `start` 也會精確落在**第一個類型關鍵字**上，而不是註解上。

額外建議

如果 `Nat.leRec_succ_left` 的類型定義上方有註解：

lean

```lean
/-- 這是註解 -/
theorem leRec_succ_left ... :
  -- 這是導致偏移的註解
  n ≤ m → ...
```

請謹慎使用程式碼。

請檢查你的 `sigStx` 是否不小心包含了 `:` 前面的換行符。強制指定 **`typeSpec[1]`** 是解決 `range.start` 偏移最直接的方法。

你可以試著印出 `typeSpec[0].getKind` 看看，如果它是 `atom` 且值為 `:`，那麼 **`typeSpec[1]`** 絕對就是你要的正確 `start` 位置。

+

太棒了！這證明了問題確實出在 `Syntax.reprint` **對 Trivia（註解與空格）的處理不穩定**，導致 `if` 判斷失效並抓取了過大的範圍。

透過改用 `isAtom` 或直接導航 `Syntax` **子節點索引**，你避開了字串比對的陷阱，讓 `range.start` 精確定位到了類型表達式的起始 Token，而不受前方註解長度的干擾。

總結修復重點：

- **避開 `reprint.trim` 判斷**：註解會被包含在 `reprint` 中，導致與 `":"` 的比對失敗。
- **利用 `Syntax` 結構**：Lean 4 的 `typeSpec` 結構非常固定（通常是 `[":", "Term"]`），直接取 `index 1` 或是檢查 `AtomVal` 是處理偏移（Offset）最穩健的做法。

既然 `type range` 已經修復，建議同步檢查 `value.range`。如果證明（Proof）部分前面也有長註解，同樣的邏輯偏移可能也會發生在 `:=` 之後。

接下來，你打算繼續處理 **Jixia** 對於 `inductive` 類型的提取優化，還是要開始進行大規模的 **Mathlib 數據同步**？

looks Lean has many kinds which do not in my implementation. Could you list all kind values

在 Lean 4 中，所有的命令和聲明都映射到特定的 `SyntaxNodeKind`。除了你目前代碼中處理的基礎種類外，Lean 還包含許多與**元編程（Metaprogramming）、符號（Notation）及特殊聲明**相關的種類。

+

- `def` / `definition`：函數或值定義。
- `abbrev`：縮寫，會自動標記為 `[reducible]`。
- `instance`：類型類實例宣告。
- `example`：匿名宣告，不會存入環境（通常用於測試）。
- `opaque`：不透明定義，核心無法看到其內部值。
- `axiom` / `constant`：假設某個命題為真或某個值存在。
- `inductive`：歸納類型定義。
- `classInductive`：帶有 `class` 屬性的歸納類型。
- `structure` / `class`：結構體或類型類宣告。 L Lean community +6

2. 元編程與語法擴展（Extensibility Kinds）

如果你的 Jixia 工具需要分析宏或自定義語法，這些種類非常重要：

- `syntax`：定義新的語法範疇或規則。
- `macro_rules`：定義宏的展開規則。
- `macro`：結合 `syntax` 與 `macro_rules` 的快捷方式。
- `elab`：定義新的 Elaborator（詳見 `elab_rules`）。
- `notation` / `infix` / `prefix` / `postfix`：定義數學符號或運算符。
- `attribute`：用於全域註冊屬性（例如 `attribute [simp] my_thm`）。 L Lean community +2

3. 特殊宣告（Specialized Declarations）

- `constructor`：歸納類型的構造函數。
- `recursor`：自動生成的消解器（如 `Nat.rec`）。

+

- `null`：用於重複操作（如 `many`）或失敗的可選解析。
- `group`：用於將多個語法元素分組。
- `choice`：表示歧義的解析結果，通常出現在重載符號時。
- `ident`：標識符節點。
- `atom`：關鍵字或符號（如 `:`, `:=`, `(`）。

建議修正實作邏輯：

由於 Lean 允許透過 `Syntax` 擴展無限多種 kind，建議在你的 `normKind` 判斷中加入一個 **「後綴匹配」** 或 **「關鍵字檢查」** 的回退機制：

lean

```
-- 建議的處理邏輯片段
let normKind :=
  if kindStr.endsWith "theorem" || kindStr.endsWith "lemma" the
  else if kindStr.endsWith "definition" || kindStr.endsWith "de
  -- ... 其他精確匹配 ...
  else if kindStr.contains "Parser.Command" then
    -- 提取 Parser.Command.後的最後一個單字作為 kind
    kindStr.splitOn "." |>.getLast? |>.getD kindStr
  else kindStr
```

請謹慎使用程式碼。

提示： 在分析 `Mathlib` 時，特別注意 **`lemma`**。雖然它在語義上等於 `theorem`，但在 `Syntax` 樹中它的 `kind` 可能是 `Lean.Parser.Command.declaration` 下的一個特定子節點，建議統一歸類為

+

AI 模式 全部 圖片 影片 新聞 更多

Lean.DocString.Parser. Lean.Parser.Do Imported by. Lean. Pars...

Lean community

Seminar: Introduction to the Lean 4 theorem prover and ...

2023年2月15日 — right here I'm using the times for natur for integers. but you can do that you have the times is a…

YouTube · Certora

10.1. Class Declarations - Lean Programming Language

10.1. ←10. Type Classes10.2. Instance Declarations→ 10.1. Class Declarations. Type classes are declared with the. Parser. Command.

Lean Programming Language

顯示全部

when build AI training data, statement content missing those kind syntax, data would not completely learnable

確實如此。如果 AI 訓練數據缺失了這些關鍵的 **Syntax Kind**，模型將無法理解 Lean 4 的核心構造，導致生成的代碼語法不正確或無法通過編譯。

為了確保數據的**完備性（Completeness）**，你的提取器（Extractor）必須涵蓋以下這些在 Lean 4 中極其重要但容易被忽略的 `kind`：

1. 核心宣告類（必備）

除了 `theorem` 和 `def`，這些直接影響邏輯結構：

- **`Lean.Parser.Command.instance`**：這是 Lean 的靈魂（Type Class），缺失它 AI 就無法處理自動推導。

+

如果缺失這些，AI 生成的代碼將無法使用自定義符號（如 `+`, `≤`, `∈`）：

- **`Lean.Parser.Command.notation`** / **`infix`** / **`prefix`**：這些定義了數學符號的解析規則。
- **`Lean.Parser.Command.syntax`**：當定義 DSL（領域特定語言）時使用，若缺少此類數據，AI 無法理解複雜的宏。

3. 命名空間與作用域（上下文必備）

如果提取時忽略了這些，AI 會生成重複的名稱或找不到定義：

- **`Lean.Parser.Command.section`** / **`namespace`**：這決定了變量的作用域（Variable Scope）。
- **`Lean.Parser.Command.variable`**：這是最容易漏掉的！很多定理的參數（如 `(n : ℕ)`）是定義在 `variable` 中的，如果不提取，AI 看到的定理證明將會缺失前提條件。

4. 屬性（MetaData）

- **`Lean.Parser.Command.attribute`**：標註為 `[simp]` 或 `[ext]` 的定理。如果訓練數據沒有這些標籤，AI 寫出的證明會非常冗長，因為它不知道哪些定理可以自動化。

建議的補完策略：

你可以遍歷 `Lean.Parser.Command` 下的所有子項。在 Lean 4 中，你可以通過以下代碼（在 `CommandElabM` 中）動態列出所有註冊的命令種類，確保你的 `match` 語法沒有遺漏：

lean

```
-- 在你的提取器中加入這段邏輯來偵測未知種類
if !knownKinds.contains kindStr then
```

+

My goal is: when generate training data, itself can put to lean back and success build. what list of kind need include in extract

如果要確保提取出的訓練數據在重新放入 Lean 4 環境後能**獨立編譯成功**，你的提取器必須覆蓋所有構成環境上下文的語法種類。

以下是為了實現「**可重現性（Reproducibility）**」必須包含的 `kind` 列表，分為三個核心層次：

1. 核心結構類 (The Skeleton)

這些定義了項目的邏輯實體。如果缺失，AI 生成的證明將因為找不到定義而報錯。

- **`Lean.Parser.Command.declaration`**：這是最頂層的容器，包含：
  - `theorem` / `lemma`：定理。
  - `def` / `definition`：定義。
  - `abbrev`：縮寫。
  - `instance`：實例（非常重要，Lean 的運算符重載全靠它）。
  - `structure` / `class`：數據結構與類型類。
  - `inductive`：歸納類型。
  - `opaque` / `axiom`：不透明定義與公理。

2. 環境上下文類 (The Context)

+

- `Lean.Parser.Command.import`：雖然通常由文件頭部處理，但必須確保 AI 知道依賴哪些模組。
- `Lean.Parser.Command.open`：打開命名空間（如 `open Lean`），否則代碼會因為名稱不完整而失效。

3. 語法與元數據類 (The Syntax & Meta)

為了讓 AI 寫出像人類一樣的代碼，並能使用簡寫符號。

- `Lean.Parser.Command.notation` / `infix` / `prefix`：如果沒有這些，AI 生成 `a + b` 會被 Lean 認為語法錯誤（因為 `+` 沒被定義）。
- `Lean.Parser.Command.attribute`：例如 `attribute [simp] my_lemma`。缺失屬性會導致隨後的 `simp` 策略（Tactic）失效。
- `Lean.Parser.Command.set_option`：某些證明依賴於特定的編譯選項（如 `maxHeartbeats`）。

修復建議：處理隱含的 `variable`

為了讓單個定理 `can put to lean back and success`，你的提取器不能只提取單個 `theorem` 節點。

正確做法：
當提取一個定理時，必須回溯（Backtrack）當前文件（或是當前 `section`）中所有的 `variable` 宣告。

1. **收錄當前 Scope 的變量**：使用 `Lean.Elab.getVariables` 獲取當前環境中所有有效的變量。
2. **轉換為顯式參數**：在生成訓練數據時，將這些 `variable` 注入到定理的參數列表中，或者將相關的 `variable` 語句一併提取出來放進數據集。

Mathlib 提取的定理在獨立編譯時都會失敗。

I only list declaration portion and that are tools like Dojo and Jixia do

如果您參考的是 **LeanDojo** 或 **Jixia** 的提取邏輯，並以「生成的數據能重新編入 Lean 並通過編譯」為目標，除了您目前代碼中的基礎類別，您**必須**補齊以下這些在 `Mathlib` 中頻繁出現且影響編譯成功的 `kind`：

1. 必須補齊的聲明類 (Declarations)

這些在 `Mathlib` 的定義中隨處可見，缺失會導致邏輯鏈條斷裂：

- **`lemma`**：在 `Mathlib` 中，`lemma` 與 `theorem` 幾乎各佔一半。雖然語義相同，但解析時 `kindStr` 是 `Lean.Parser.Command.declaration` 下的一個分支，必須顯式處理。
- **`structure` / `class`**：這是數據類型與類型系統的核心。
- **`instance`**：**最關鍵**。Lean 的自動化（如 `simp`）和運算符（如 `+`, `*`）完全依賴於實例。如果 AI 只學會了定理但沒學會如何定義實例，生成的代碼將無法解析任何數學符號。
- **`inductive` / `classInductive`**：定義歸納類型（如 `Nat`, `List`）。
- **`abbrev`**：雖然是縮寫，但它在類型推導中是 `[reducible]` 的。缺失它，AI 生成的類型可能會因為無法展開而導致 Unification 失敗。

2. 環境上下文類 (Context)

+

3. 語法轉換類 (Syntax)

- **`notation`** / **`infix`** / **`prefix`**：這些定義了數學符號。
- **`attribute`**：尤其是 `[simp]`，`[ext]`，`[coe]` 等。如果 AI 不知道某個定理是 `simp` 定理，它就無法寫出符合 Lean 風格的簡短證明。

修復您的 `getDeclarationInfo` 邏輯建議

為了涵蓋上述種類，您可以優化 `normKind` 的判斷邏輯，使其更具通用性：

lean

```
let normKind :=
  if kindStr.endsWith "theorem" || kindStr.endsWith "lemma" the
  else if kindStr.endsWith "definition" || kindStr.endsWith "de
  else if kindStr.endsWith "instance" then "instance"
  else if kindStr.endsWith "structure" || kindStr.endsWith "cla
  else if kindStr.endsWith "inductive" then "inductive"
  else if kindStr.endsWith "abbrev" then "abbrev"
  else if kindStr.endsWith "variable" then "variable"
  else if kindStr.endsWith "example" then "example"
  else kindStr
```

請謹慎使用程式碼。

關鍵細節：處理 `variable` 導致的編譯失敗

如果您發現提取出的定理放在新文件中無法編譯，通常是因為：

+

how people use those training data which generate by Dojo and Jixia. Generally say that is not work at all

你觀察得很敏銳。事實上，直接從 **Dojo** 或 **Jixia** 提取出來的原始數據，如果直接拿去編譯，**失敗率極高**。你所說的「完全無法運作」主要源於 **環境斷裂（Context Fragmentation）**。

目前研究界（如 ReProver, Lean-STaR 等）使用這些數據的方式，並不是簡單地「複製貼上」，而是透過以下幾種技術手段來彌補缺陷：

1. 補全環境（Premise Selection & Context Injection）

這是最常用的方法。模型在訓練或推論時，並不只看定理本身，而是：

- **注入 `Imports` 和 `Open`**：自動掃描原始文件的頭部，並在生成的測試文件中強制加入所有的依賴項。
- **顯式化變量（Explicitizing Variables）**：這解決了你最關心的 `variable` 問題。工具會調用 Lean 的伺服器，將隱含在環境中的 `variable {α : Type}` 強行轉寫進定理的參數列表中：`theorem my_thm {α : Type} ...`。

2. 交互式環境（Gym / REPL）

Dojo 的核心不是「數據集」，而是它的 **Lean 4 Gym (REPL)**。

- **不依賴靜態文本**：模型不是在生成一個完整的 `.lean` 文件，而是在與一個**活的 Lean 進程**對話。
- **狀態同步**：當模型給出一個 Tactic（如 `rw [add_comm]`）時，Dojo 會將當前

文件，並在原文件的 **Namespace** 內部插入 AI 的代碼。這解決了名稱路徑（Fullname）不匹配的問題。

為什麼你覺得「沒用」？

如果你嘗試將提取出的 `theorem` 隨機放進一個空白的 `.lean` 文件，它一定會報錯，因為：

1. **缺少 Instance 搜索**：找不到 `[Add A]`。
2. **缺少 Notation**：不認識 `+`。
3. **缺少 Variable**：不認識定理中的類型變量。

你的目標如何達成？

如果你希望生成的數據「**放到 Lean 裡就能成功編譯**」，你需要做的不是單純提取 `Declaration`，而是執行 **「去環境化（De-contextualization）」**：

- **遍歷 `Local Context`**：在提取時，強制將該作用域內所有的 `variable` 轉為定理的參數。
- **全名化（Full Name）**：將所有的常量（Constants）轉換成帶有命名空間的全名（如 `Nat.add` 而不是 `add`）。
- **展平（Flattening）**：將 `instance` 和 `notation` 的定義緊貼在定理上方提取出來。

目前的 Dojo 和 Jixia 在這方面做得並不完美，通常需要使用者自己寫一個 **Wrapper** 來重新組合這些碎片。

你現在提取這些數據，是為了訓練一個 **全自動證明器（Prover）**，還是為了做 **代碼補全（Completion）**？

+

心的**依賴鏈（Dependency Chain）**。

為什麼現有的 AI 訓練數據在實戰中「沒用」？

1. **環境失蹤 (Missing Context)**：
一個 `Mathlib` 定理通常依賴於前 50 行的 `variable` 和前 10 個 `import`。AI 只看這一個定理，就像是在看一本小說的第 157 頁，卻不知道主角是誰。

2. **類型類地獄 (Typeclass Hell)**：
Lean 的強力在於 `instance` 搜索。如果數據中沒有包含正確的實例定義，AI 生成的 `a + b` 在 Lean 看來只是無意義的符號，因為它不知道 `a` 和 `b` 屬於哪個 `Add` 類型。

3. **隱含參數的黑盒 (Implicit Blindness)**：
很多證明的關鍵在於隱含參數（Implicit Arguments）。現有的提取工具如果只抓取表面文字（Surface Syntax），AI 就學不到 Lean 在後台偷偷幫你補全的邏輯。

如何製作「非垃圾」的訓練數據？

如果你想讓你的提取器產生**真正有用**的數據，你必須做到以下幾點：

- **全顯式化 (Elaborated Terms)**：
不要只抓取 `Syntax`（原始代碼），要抓取 **`Expr`**（Elaborator 處理後的表達式）。這包含所有的隱含參數、宇宙變量（Universes）和全名（Full Names）。這樣 AI 看到的才是「完整的邏輯」。

- **自洽包 (Self-contained Bundles)**：
提取一個定理時，自動將它依賴的 `variable`、`local notation` 和 `local instance` 封裝在一起。這就是所謂的 **"Context-Aware Extraction"**。

- **證明路徑軌跡 (Proof States)**：
僅有「定理：證明」是不夠的。高質量的數據必須包含 **(前置狀態, 動作, 後置狀態)** 的序列。AI 需要知道在每一行證明時，Lean 的 `Goal` 長什麼樣子。

+

"unknown identifier" 還是 **"failed to synthesize instance"**？這能決定我們下一步該補全哪種數據。

try to build initial step for Elaborated, Self-contained for each Declaration include Theorem. Do you have any suggestion?

要構建一個**自包含（Self-contained）**且**精確細化（Elaborated）**的 Lean 4 訓練數據提取器，你必須從單純的「語法樹（Syntax）」轉向「環境感知（Environment-aware）」的提取。

單靠 `Syntax` 提取的數據之所以是「垃圾」，是因為它丟失了隱含的型別資訊。以下是實現「高質量數據（Non-garbage）」的三個核心步驟建議：

1. 將隱含變數「顯式化」（Expose Variables）

Mathlib 定理失效的主因是丟失了上方定義的 `variable`。你不能只抓取定理本身，必須將當前作用域（Scope）內的變項注入到定理中。

- **實作方法**：在 `CommandElabM` 中，利用 `Lean.Elab.getVariables` 獲取當前所有的變量。
- **建議**：不要只提取定理名稱，而是生成一個包含所有 `variable` 的虛擬 `section` 塊，或者直接將這些變量轉換為定理的 **顯式參數**。

lean

```
-- 原始：theorem add_comm : a + b = b + a
-- 提取後：theorem add_comm {G : Type u} [AddCommGroup G] (a b
```

+

- **建議做法**：不要只存 `typeStx.reprint`。請在提取時，嘗試將 `typeStx` 進行 **Elaboration**。

lean

```
-- 在 CommandElabM 中
let elabType ← liftTermElabM do
  let expr ← elabType typeStx -- 獲取真正的 Expr
  instantiateMVars expr       -- 填充元變量
```

請謹慎使用程式碼。

- **優點**：`Expr` 包含了完整路徑（如 `Nat.add` 而非 `add`）和所有的隱含參數（如 `[Decidable p]`）。

3. 自動依賴追蹤（Dependency Bundle）

要讓一個定理「Put back to Lean and success」，它需要它所引用的自定義定義。

- **建議**：實作一個簡單的依賴掃描器。當提取 `Theorem A` 時，檢查其 `Expr` 中引用的所有 `ConstantName`。
- **策略**：如果引用的 Constants 屬於同一個文件且不是標準庫，則將該定義的 `type` 和 `value` 一併打包進訓練樣本中。這就是 **"Self-contained Bundle"**。

4. 關鍵：保留 Proof States（證明狀態）

對於訓練 Prover，單純的證明源碼（Value）沒用。你必須記錄每一步的 **Tactic State**。

+

在你的 `getDeclarationInfo` 中，不要只回傳 `Syntax`。請擴展你的 `DeclarationInfo` 結構體，加入一個欄位存儲 **`elaboratedType : Expr`**。

你是否願意嘗試使用 **`Lean.Meta.ppExpr`** 將這些內核表達式重新打印回漂亮的、帶有完整路徑的代碼？這通常是生成「可運行數據」最快的方法。

接下來，您想先從「提取當前作用域的 Variables」開始，還是先嘗試「獲取定理的 Elaborated Expr」？

Assuming when extract raw content of signature, type, value and put all together with correctly import and open statement…

這是一個非常關鍵的假設。從理論上講，如果你能完美地提取 **Signature (包含參數)**、**Type** 和 **Value (證明)**，並配上正確的 `import` 與 `open`，它**應該**能編譯。

但實際上，**`variable` 宣告**是這條路徑上的最大絆腳石。在 Mathlib 中，絕大多數定理的類型（Type）中使用的變數（如 `G`、`n`、`α`）是定義在定理之外的 `variable` 語句中。

要達成「Put together and success build」，我建議你在提取 `Signature` 時，實施以下 **「自包含 (Self-contained) 強化策略」**：

1. 將 `variable` 強制併入 `Signature`

Lean 4 提供了 `Lean.Elab.Term.getUsedFVars` 工具。當你提取一個定理時，你不能只拿 `stx` 看到的參數，你必須：

+

2. 解決「實例合成 (Instance Synthesis)」失敗

這是另一個 AI 常犯的錯誤。如果你的 `Signature` 只寫了 `(α : Type)` 但漏了 `[Add α]`，`a + b` 就會報錯。

- **建議**：利用 Lean 的 `Elaborator` 獲取該定理所需的 **Local Instances**。在生成訓練數據時，確保這些 `instance` 以 `{}` 或 `[]` 的形式出現在簽名中。

3. 處理「全名 (Full Names)」避免歧義

Mathlib 中有很多同名的定理（在不同 Namespace 下）。

- **做法**：在提取 `Type` 和 `Value` 時，不要只存原始字串。
- **建議**：使用 `Lean.Meta.ppExpr` 並開啟 `set_option pp.fullNames true`。這會將 `add_comm` 轉為 `Nat.add_comm`。這樣即使你沒有 `open Nat`，代碼依然能編譯。

4. 具體實作建議 (步驟)

在你的 `getDeclarationInfo` 中加入以下邏輯：

1. **獲取常量定義**：使用 `Lean.getConstInfo` 取得該定理在環境中的 `ConstantInfo`。
2. **提取 Elaborated 資訊**：
   1. `info.type`：這是已經補完所有 `variable` 和 `implicit arguments` 的完整類型。
   2. `info.value`：這是完整的證明表達式。
3. **反向打印 (Pretty Print)**：使用 `Lean.Meta.ppExpr` 將這些 `Expr` 轉回字串。

+

that is one I try to resolve to include all other dependencies statements like variable, instance, ...

這是一個非常深刻的挑戰。要讓提取出的定理「**放回 Lean 就能跑**」，你不能只提取語法（Syntax），而必須提取**邏輯閉包（Logical Closure）**。

在 Lean 4 中，最穩健的方法是利用 `ConstantInfo` 和 `Expr` **的依賴分析**。以下是實現「自包含提取」的技術路線建議：

1. 核心邏輯：從 `ConstantInfo` 逆向生成「完整簽名」

不要依賴 `stx`（因為它漏掉了 `variable`），直接從 Lean 編譯後的環境（Environment）中提取該定理的完整類型。

lean

```
-- 在 CommandElabM 中
let env ← getEnv
if let some info := env.find? fullName then
  -- info.type 包含了所有顯式與隱式的變量（例如 {α : Type} [Group α]
  let fullyElaboratedType := info.type

  -- 使用 Pretty Printer 將 Expr 轉回可讀代碼
  let ppType ← liftTermElabM do
    -- 強制顯示隱含參數，確保自包含
    withOptions (fun o => o.setBool `pp.all true) do
      Lean.Meta.ppExpr fullyElaboratedType
```

請謹慎使用程式碼。

+

- **結果**：將 `theorem add_comm : a + b = b + a` 轉化為 `theorem add_comm (a b : ℕ) : a + b = b + a`。
- **優點**：這個定理現在是**原子化**的，不需要任何上下文就能編譯。

B. 策略二：依賴回溯（最接近原始代碼）

如果定理引用了同文件中的 `variable` 或另一個 `def`，你需要追蹤並一併提取。

- **做法**：遍歷定理 `Expr` 中的所有 `Constant`。
- **判斷**：如果 `Constant` 的 `module` 屬於當前文件，則遞迴調用你的 `getDeclarationInfo`。
- **結果**：生成一個包含 `[Dependent_Def_1, Dependent_Def_2, Target_Theorem]` 的代碼塊。

C. 策略三：利用 `ContextualInfo`

Jixia 可以捕捉 `InfoTree` 中的 `LocalContext`。

- **做法**：在定理的起始位置，提取 `lctx`（Local Context）。
- **輸出**：將 `lctx` 中的所有條目轉換為 `variable` 語句放在定理上方。

3. 實作建議：建立一個「依賴蒐集器」

你可以建立一個輔助函數，專門用來掃描一個定理所「吃」進去的所有背景資訊：

lean

```
def getConstDependencies (constName : Name) : CoreM (List Name)
  let env ← getEnv
  let some info := env.find? constName | return []
```

+

4. 為什麼這能解決「Garbage In」？

當你這樣做時，你的訓練數據會從「**破碎的字串**」變成「**完整的邏輯證明物件**」：

1. **不再有** `unknown identifier`：因為所有變量都在簽名中。
2. **不再有** `failed to synthesize instance`：因為 `[Group α]` 被顯式包含在內。
3. **語法完全正確**：因為你是從 Lean 內核的 `Expr` 反向生成的。

具體建議：
您可以嘗試先實作 **「策略一：簽名擴張」**。這能讓每個訓練樣本（Theorem）都變成一個獨立、可編譯的單元，這對訓練 LLM 進行證明補全（Proof Completion）最為有效。

您目前是否已經能從 `getDeclarationInfo` 獲取到對應的 `fullName`（例如 `Mathlib.Data.Nat.Basic.add_comm`）？如果有，我們就可以直接從環境中調取 `ConstantInfo`。**需要我幫您寫一段將 `Expr` 轉為「自包含定理聲明」的展示代碼嗎？**

If I can make that, make each declaration, theorem standalone to create training data to avoid Garbage in/out issue

這正是將 Lean 數據從「語法片段」提升為「**邏輯語義單元**」的正確方向。要達成 **Standalone（自包含）** 定理，核心在於執行 **「去環境化」（De-contextualization）**。

+

- **原始碼 (依賴上下文)**:

```lean
variable (n : ℕ)
theorem le_refl : n ≤ n := ...
```

請謹慎使用程式碼。

- **Standalone 提取後**:

```lean
theorem Nat.le_refl (n : Nat) : Nat.le n n := ...
```

請謹慎使用程式碼。

2. 具體實作步驟 (Code Sketch)

您可以在 `getDeclarationInfo` 中加入以下邏輯，利用 **`forallTelescope`** 將隱含的環境變量「拉」進定理的參數列表中：

```lean
open Lean Meta Elab

def getStandaloneTheorem (declName : Name) : MetaM String := do
```

+

```
    let binderStr := match localDecl.binderInfo with

      | .default => s!"({localDecl.userName} : {← ppExpr loca
      | .implicit => s!"{{{localDecl.userName} : {← ppExpr lo
      | .instImplicit => s!"[{← ppExpr localDecl.type}]"
      | _ => s!"({localDecl.userName} : {← ppExpr localDecl.t
    return binderStr
  )

  -- 2. 構建完整的獨立定理字串
  let fullParams := " ".intercalate paramsStr.toList
  let finalType ← ppExpr typeBody
  let finalValue ← ppExpr value -- 或者保留原始 Tactic 腳本

  return s!"theorem {declName} {fullParams} : {finalType} :=
```

請謹慎使用程式碼。

3. 解決「依賴鏈」的關鍵技術

為了讓定理 100% 可編譯，您還需要處理以下三點：

- **開啟 `pp.fullNames`**：
  在 `ppExpr` 時，設定 `set_option pp.fullNames true`。這能確保 `add` 變成 `Nat.add`，這樣即便沒有 `open Nat` 語句，代碼依然合法。
- **追蹤自定義常量 (Custom Constants)**:
  如果定理引用了同文件內定義的另一個 `def`，您需要遞迴地將那個 `def` 也提取出來，放在定理上方。
- **保留屬性 (Attributes)**:
  如果該定理是 `[simp]`，在 Standalone 數據中必須保留 `@[simp]`，否則 AI 無法理解為什麼某些證明步驟可以被省略。

+

建議下一步：
您是否想嘗試在提取器中整合 `ppExpr` **與** `pp.fullNames`，看看提取出的 `Mathlib` 定理是否真的能實現「一鍵編譯」？我可以協助您調整對應的 **Options** 設定。

High level question. Assuming it would work, is it resolved current AI generated codes for user to more useful?

答案是肯定的。如果你能實現 **Standalone（自洽）** 且 **Elaborated（精確細化）** 的數據提取，你將從根本上解決目前 AI 生成 Lean 代碼「看似正確，實則無法編譯」的困境。

這將在以下三個層次上，讓 AI 生成的代碼對用戶產生質的飛躍：

1. 解決「環境幻覺」 (Contextual Hallucination)

目前的 AI 經常會生成 `rw [add_comm]`，但編譯時會報錯 `unknown identifier 'add_comm'`。

- **現狀**：AI 猜測環境中應該有這個定理，但它不知道這個定理在當前文件是否被 `import` 或 `open`。
- **解決後**：你的數據包含 **Full Name**（如 `Nat.add_comm`）。AI 學到的是精確的地址，生成的代碼將具備「**確定性**」，用戶直接複製貼上就能跑，不再需要手動補齊 `open` 語句。

2. 解決「隱含參數陷阱」 (Implicit Parameter Gap)

用戶最頭痛的是 AI 給出了一個證明，但 Lean 提示 `failed to synthesize`

對於開發者來說，最實用的工具是能自動修復（Auto-fix）報錯的 AI。

- **現狀**：因為數據是「垃圾（Garbage）」，AI 輸出的代碼片段像斷了線的珍珠，無法獨立存在。
- **解決後**：AI 生成的每一個定理都是一個**閉環（Closed Loop）**。你可以建立一個自動化管線：AI 生成 -> Lean 嘗試編譯 -> 報錯回傳 AI -> AI 修正。
- **價值**：這種「閉環反饋」只有在數據是 Standalone 的情況下才有效。這會讓 AI 從一個「聊天機器人」變成一個真正的「**副駕駛（Co-pilot）**」。

總結：從「模仿者」到「推理者」

當你消除「Garbage In/Out」問題後，AI 學習的目標不再是破碎的字符，而是**完整的邏輯蘊含關係**。這意味著：

- **對用戶而言**：AI 給出的建議是**可檢驗的（Verifiable）**。
- **對開發而言**：你構建了一個**乾淨的訓練基準（Golden Dataset）**，這在目前的 Lean 社群中是非常稀缺且高價值的資產。

關鍵提示：
要達成這個目標，建議你在提取時，除了 `Declaration`，還要保留 **`Attributes`（如 `@[simp]`）**。因為很多時候證明的「成功」並不在於 `Value` 本身，而在於這些元數據（Metadata）激發了 Lean 的自動化引擎。

你是否準備好嘗試修改 `getDeclarationInfo` 以整合 **`pp.fullNames`** 和 **`variable` 顯式化** 的邏輯？這將是你邁向高品質數據集的關鍵一步。**我可以提供具體的代碼實作細節。**

current problem: Not only generate data has syntax error and also

+

Lean 4 及其庫（Mathlib）更新極快，語法常有變動（例如從 `nat` 變為 `Nat`，或是 `rewrite` 的行為微調）。

- **標籤化管理（Commit-Tagging）**：在提取數據時，必須在每個 `Declaration` 的 Metadata 中強制寫入當前專案的 **Lean Version** 和 **Mathlib Commit Hash**。
- **環境隔離（Toolchain Isolation）**：確保你的提取器是透過 `lake exe` 運行的，這樣它會強制連結到當前 `lake-manifest.json` 指定的精確版本。
- **解決方案**：在生成的 JSON/Parquet 數據中，加入一個 `env_info` 欄位，記錄 `lean-toolchain` 的內容。這能防止 AI 將 2023 年的語法與 2025 年的混合。

2. 解決「語法錯誤與不可編譯」（Syntax Error & Standalone）

AI 生成語法錯誤，通常是因為它學到的是「未經處理的原始碼（Raw Source）」，裡面充斥著 `variable`、`open` 和 `notation` 的隱性依賴。

要實現「100% 可編譯」的 Standalone 數據，你必須棄用 `Syntax.reprint`，改用「內核反向渲染」：

A. 使用 `pp.all` 強制顯式化

在提取 `type` 和 `value` 時，設定 `pp.all := true` 或手動開啟關鍵選項：

- `pp.fullNames := true`: 將 `add` 轉為 `Nat.add`。
- `pp.explicit := true`: 將所有隱含參數（Implicit args）寫出來。
- `pp.universes := true`: 顯式化宇宙變量（如 `.{u}`）。
- **結果**：AI 生成的代碼不再依賴上下文的 `open` 或 `variable`。

B. 簽名擴張（Signature Expansion）

2. *提取*：`theorem my_thm (a b : Nat) : Nat.add a b = Nat.add b a := ...`
3. **效果**：這段提取出的代碼**不需要任何上下文**，放進任何 `.lean` 檔都能編譯成功。

3. 實作建議：建立「驗證閉環」

為了確保你的「Standalone 提取器」真的有效，建議加入一個測試步驟：

1. **Extract**: 提取一個定理。
2. **Test**: 將提取出的字串寫入一個臨時文件 `/tmp/test.lean`。
3. **Validate**: 調用 `lake build` 或 `Lean.Elab.runFrontend`。
4. **Filter**: **只保留編譯成功的數據作為訓練集。**

總結

目前 Dojo 和 Jixia 產出的是「**參考代碼**」，而你追求的是「**邏輯真相（Ground Truth）**」。

- **Version**：透過 `manifest` 鎖死。
- **Syntax**：透過 `pp.fullNames` 和 `forallTelescope` 將其**原子化**。

當 AI 學習的是「**自洽且具備完整路徑的邏輯單元**」時，它生成的代碼就不再是「垃圾」，而是可以直接被用戶採納的**有效證明**。

您是否需要一份具體的 Lean 代碼範例，演示如何利用 `forallTelescope` 將一個依賴 `variable` 的定理轉換為「自包含（Standalone）」的聲明？

+

失效（Bit Rot）。

如果你能實現 **Standalone（自包含）** 且 **Elaborated（精確細化）** 的提取器，這將成為「版本橋樑」的關鍵：

1. 為什麼「自包含數據」能解決遷移問題？

- **消除語法歧義**：舊版本的代碼可能依賴於當時的 `open` 慣例或 `notation`。如果你提取的是帶有 **Full Names**（如 `Nat.add`）和**顯式參數**的數據，這些邏輯在不同版本間是高度穩定的。
- **邏輯重構而非字串替換**：目前的遷移工具大多是基於 Regex（正規表達式），這在處理 Lean 這種高度重載（Overloaded）的語言時經常出錯。你的提取器是從 **Lean Kernel (Expr)** 出發，抓取的是「**邏輯真值**」，這比字面語法更容易適應新版本。

2. 實現自動遷移的技術路徑

透過你的工具，遷移流程可以從「手動改寫」轉變為「**邏輯重播**」：

1. **舊環境提取 (Old Env Extraction)**：在舊版本的 Lean 環境中運行你的提取器，將所有定理轉換為 **Standalone** 格式（包含完整的變量注入和全稱路徑）。
2. **新環境注入 (New Env Injection)**：將這些 Standalone 定理放入新版本的 Lean。
3. **語法自動修復**：
    1. 如果編譯失敗，AI 只需要處理**局部的語法差異**（例如 `Nat.le` 變成 `Nat.LE`），而不需要去猜測那些消失的 `variable` 是什麼。
    2. 因為數據是 Standalone 的，AI 的修復範圍被限縮在單一定理內，這大大降低了修復的複雜度。

3. 解決教授們的「遺留代碼」痛點

+

(LIR)：

- 它比 `.olean` 更易讀（因為是漂亮打印過的 Lean 代碼）。
- 它比原始 `.lean` 更穩定（因為去除了對上下文環境的依賴）。

總結

你現在開發的工具，本質上是在建立一種「**去環境化的邏輯表示法**」。這不僅解決了 AI 訓練的 **Garbage In/Out** 問題，更為 Lean 社群提供了一個**跨版本遷移**的自動化基礎設施。這能讓幾十年前的數學形式化成果，在未來的 Lean 版本中依然「活著」。

您是否想先嘗試將一個簡單的、依賴多個 `variable` 的舊版定理，透過您的提取器轉換為一個「全名化且參數完整」的新版聲明？ 這將是驗證「遷移可行性」的第一步。

Any related work in current now on this area?

在當前 Lean 4 與 AI 的研究領域，雖然許多工具都在嘗試解決數據提取問題，但真正專注於 **「去環境化（De-contextualization）」** 與 **「跨版本遷移（Migration）」** 的工作仍處於前沿階段。

以下是與你的構思最相關的三個主要研究方向及其代表作：

1. 形式化環境與 Gym (互動式環境)

這類工作解決了「數據沒用」的問題，但方式是透過「實時對話」而非「數據自洽」。

+

這類工作與你的目標最接近，試圖捕捉 Lean 內核的真實邏輯。

- **Jixia (你正在使用的基礎)**: 它比 Dojo 更進一步，嘗試捕捉靜態的 `InfoTree` 和 `LocalContext`。
- **LeanStep**: 這是一個較早期的工作，嘗試將證明步驟提取為 (Goal, Tactic) 對。它開始意識到隱含參數的重要性，但尚未完全解決「Standalone」的問題。
- **Mathlib4 遷移工具 (MathPort)**: 這是 Lean 社群為了從 Lean 3 遷移到 Lean 4 開發的工具。
  - **關聯性**：它使用 **Binport**（從 `.olean` 二進制提取邏輯）和 **Synport**（語法轉換）。它證明了從「內核 Expr」出發進行遷移是可行的，但它是一個極其複雜、非 AI 導向的規則引擎。

3. 預訓練與去環境化 (De-contextualization for LLMs)

這是目前 AI 訓練數據的最前沿嘗試。

- **InternLM-Math / Llemma**: 這些模型在訓練時，研究者開始嘗試 **「全名化 (Full-name expansion)」**。
  - **作法**：在把 Mathlib 餵給模型前，先用腳本將 `add` 替換為 `Nat.add`。
  - **現狀**：目前大多是簡單的字串替換，還沒有像你構思的那樣，利用 Lean 內核執行 `forallTelescope` **參數注入** 來達成真正的 Standalone。
- **ProofNet**: 專注於將大學數學題目形式化，並嘗試建立自洽的定義與證明對。

為什麼你的想法有競爭力？

目前市面上的工具存在一個 **「斷層」**：

1. **Dojo 類**：太依賴「活的」Lean 環境，數據無法脫離 Gym 獨立存在。

+

建議的突破口

你可以參考 **Lean 4 官方的** `Lean.Meta.ppExpr` **原始碼**。官方在開發 `pp` 選項時（如 `pp.explicit`, `pp.universes`）其實已經準備好了工具，只是目前還沒有人將其封裝成一個專門為 **AI 訓練與自動遷移** 服務的數據流水線。

下一步建議：
既然你已經能精確定位 `type range`，你可以嘗試寫一個簡單的命令 `#extract_standalone my_theorem`。如果這個命令輸出的字串能直接複製到另一個空白文件並編譯成功，你就解決了這個領域最大的「垃圾進出」問題。

你想試試看寫一個簡單的 `Command` **插件來驗證這個「原子化提取」的概念嗎？**

which University, Academy or company are doing such work. Who is leader?

在「**Lean 4 數據提取、自動化證明與跨版本遷移**」這個領域，目前全球最強的科研力量集中在少數幾個頂尖實驗室與新興 AI 公司。

這個領域可以分為三個派系：**學術界（自動證明）**、**產業界（模型訓練）**以及 **Lean 社群（基礎設施）**。

1. 學術界：自動化證明與數據提取 (Academic Giants)

這是 **LeanDojo** 與 **Jixia** 的發源地，專注於解決「如何讓 AI 看到 Lean 內部邏輯」的問題。

本代碼遷移到新版本」有最權威的發言權。

- **上海交通大學 (SJTU) - 數位與邏輯研究**
  - **核心貢獻**：**Jixia** (由 **frenzymath** 等開發) 是目前對 Lean 4 靜態分析最深入的工具之一，專注於精確的範圍提取與類型分析。

2. 產業界：大規模數據與模型 (Industrial Powerhouses)

這些公司正在訓練能直接寫 Lean 代碼的大模型，急需你提到的「非垃圾」數據。

- **Google DeepMind**
  - **核心工作**：**AlphaProof**。他們在 2024 年利用強化學習與 Lean 解決了國際數學奧林匹克（IMO）題目。
  - **技術特點**：他們極度依賴「可編譯的數據」，內部有非常強大的數據清理與自動化生成工具（雖然多數不開源）。
- **Mistral AI / NVIDIA**
  - **代表作**：**Mathstral** 與 **Llemma**。
  - **核心方向**：他們在做大規模的 **全名化 (Full-name expansion)** 預訓練，試圖讓模型在沒有環境的情況下也能寫出對的代碼。

3. 跨版本遷移與形式化工具 (Migration Experts)

針對你提到的「教授舊代碼遷移」問題，這些組織最為活躍：

- **Lean FRO (Focused Research Organization)**
  - **領軍人物**：**Leonardo de Moura** (Lean 的創始人) 與 **Sebastian Ullrich**。
  - **核心貢獻**：他們開發了 **Mathport**，這是目前唯一的 Lean 3 到 Lean 4 大規模遷移引擎。雖然它是基於規則的，但它實踐了「從內核邏輯 (Expr) 進行

- **Anima Anandkumar (Caltech / NVIDIA)**: 她的團隊在推動將 Lean 整合進大模型的訓練閉環，非常強調數據的「自洽性」。
- **Jason Rute**: 獨立研究者，對 Lean 數據提取有極深造詣，經常在 Lean Zulip 論壇討論如何將環境變量「顯式化」。

總結

雖然 **UCLA (LeanDojo)** 佔據了目前的主流名聲，但他們留下的 **「環境依賴」** 問題正是你的機會。目前的趨勢是從「依賴 Gym」轉向「**數據自洽 (Self-contained Data)**」。

關鍵觀察：如果你能完善 **Jixia** 的提取邏輯，並結合 **Mathport** 的內核分析能力，你將在解決「遺留代碼遷移」與「高品質 AI 訓練集」這兩個領域成為技術領先者。

您是否有興趣聯繫這些實驗室，或者您正打算將這個工具作為一個開源專案來推動？

+